CPSC 471 Socket Programming Project

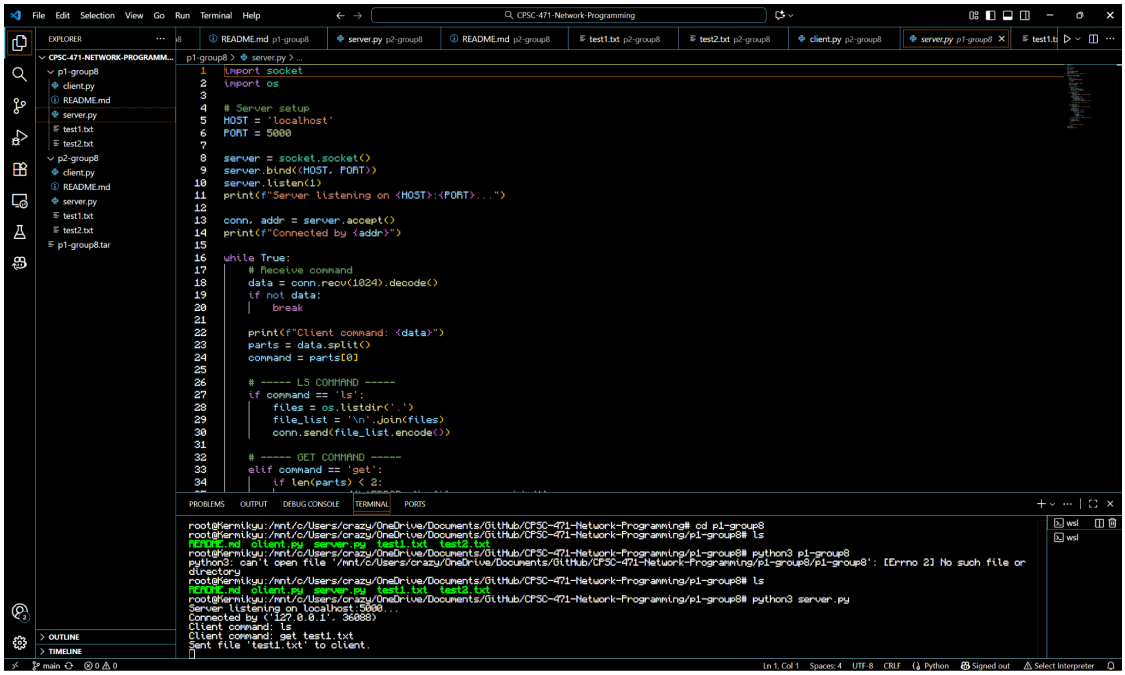
Phase 1 Progress Report

Bryan Tran | btran299@csu.fullerton.edu

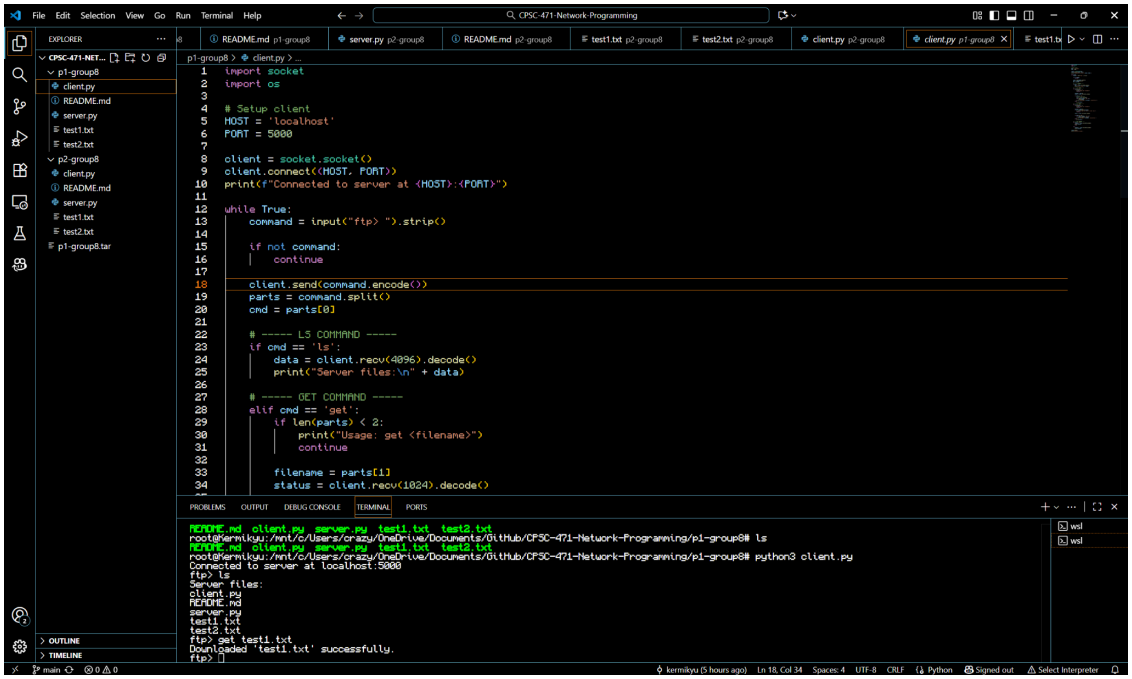
Peter Chau | Peterchauu93@csu.fullerton.edu

Current Status of Phase 1:

server.py

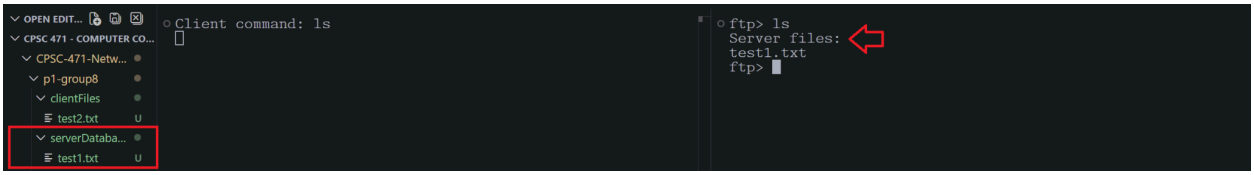


client.py



Screenshots provided separate server and client response (left - server | right - client)

LS command:



- Client's visibility only within the server's database, displaying all existing files within the 'serverDatabase' directory

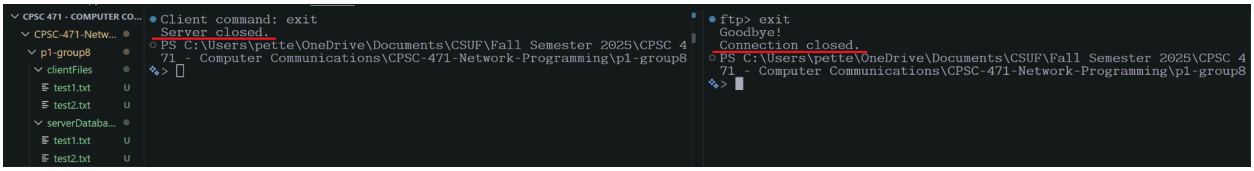
GET command:

- Client is able to grab 'test1.txt' from the server's database and download it into the client's 'clientFiles' folder

PUT command:

- Client is able to send its own file, 'test2.txt', across the socket connection, which is then verified by the server and then stored in the server's database

EXIT command:



- Once the client terminates the connection, the server subsequently sends a final 'Goodbye!' message and closes its connection alongside the client

Future outlook for Phase 2:

Both teammates share equal responsibility for the project by reviewing each other's code and testing all functionality to ensure everything works correctly and consistently. Future incorporations will involve multiprocessing capabilities on the server side to handle multiple client communication lines. Clients should still retain the ability to perform the appropriate commands established in phase 1. Incorporate a database architecture that allows for logging/monitoring of network traffic information on some storage device. Test each functionality separately and evaluate performance to resolve conflicts or communication issues before deploying onto the cloud through AWS buckets for further concurrency assessment.

Timeline:

November 15: Bryan and Peter will expand on client functionality for multi-concurrency, simulating tests with multiple clients.

November 19: Determine which method of logging is most preferred and begin implementing the database structure, aggregating data locally whenever the server is active.

November 23: Begin deployment into the AWS cloud, resolving possible run-ins with subscription and instantiating the server as well as client connections from multiple sources (local or on cloud).

November 27: Conclude with further testing and monitoring, verifying functionality and availability submission on November 30th.